

1. Explique a diferença entre **mysql2** e **mysql2/promise**.

O pacote **mysql2** utiliza callbacks para executar consultas ao banco de dados, enquanto **mysql2/promise** oferece suporte nativo a Promises, permitindo o uso de **async/await**. Isso torna o código mais limpo, organizado e fácil de manter.

Exemplo com **mysql2/promise**:

```
const mysql = require("mysql2/promise");

async function listarUsuarios() {

  const conexao = await mysql.createConnection({

    host: "localhost",

    user: "root",

    password: "",

    database: "empresa"

  });

  const [usuarios] = await conexao.query("SELECT * FROM usuarios");

  console.log(usuarios);

  await conexao.end();

}
```

2. Quais as vantagens de usar variáveis de ambiente na conexão com o banco de dados?

As variáveis de ambiente oferecem diversas vantagens:

- Protegem informações sensíveis como usuário e senha.
- Evitam deixar credenciais diretamente no código.
- Facilitam mudanças entre ambiente de desenvolvimento, testes e produção.
- Melhoram a manutenção do projeto.
- Aumentam a segurança da aplicação.

3. O que é um Prepared Statement e como ele ajuda na segurança da aplicação?

Um **Prepared Statement** é uma consulta SQL parametrizada.

Exemplo:

```
const sql = "SELECT * FROM usuarios WHERE id = ?";
```

```
const [resultado] = await conexao.execute(sql, [id]);
```

Ele impede que comandos SQL enviados pelo usuário sejam interpretados como parte da consulta, protegendo a aplicação contra ataques de **SQL Injection**.

4. Por que é recomendável usar conexões em Pool em aplicações Node.js?

O Pool de Conexões mantém várias conexões abertas e reutilizáveis.

Suas vantagens incluem:

- maior desempenho;
- redução do tempo para novas conexões;
- menor consumo de recursos;
- suporte a múltiplos usuários simultaneamente;
- melhor escalabilidade.

5. Descreva o papel do módulo **dotenv** em projetos Node.js.

O pacote **dotenv** carrega variáveis armazenadas no arquivo **.env**.

Exemplo:

Arquivo `.env`

`DB_HOST=localhost`

`DB_USER=root`

`DB_PASSWORD=123456`

`DB_NAME=empresa`

Arquivo JavaScript:

```
require("dotenv").config();
```

```
console.log(process.env.DB_HOST);
```

Isso evita expor dados confidenciais diretamente no código.

6. Qual é a estrutura básica para realizar uma consulta **SELECT** com o pacote `mysql2`?

Exemplo:

```
const mysql = require("mysql2/promise");
```

```
async function listar() {
```

```
  const conexao = await mysql.createConnection({
```

```
    host: "localhost",
```

```
    user: "root",
```

```
    password: "",
```

```
    database: "empresa"
```

```
  });
```

```
const [dados] = await conexao.query("SELECT * FROM usuarios");

console.log(dados);

await conexao.end();
}

listar();
```

7. Liste três boas práticas ao conectar o Node.js com o MySQL.

- Utilizar `mysql2/promise` com `async/await`.
- Utilizar Prepared Statements para evitar SQL Injection.
- Armazenar credenciais no arquivo `.env`.

Também é recomendado:

- utilizar Pool de Conexões;
- tratar erros com `try/catch`;
- organizar o código em módulos.

8. Explique o conceito de transações no MySQL e como ele pode ser implementado em Node.js.

Uma transação permite executar várias operações como uma única unidade.

Caso alguma operação falhe, todas as alterações podem ser desfeitas através do **ROLLBACK**.

Exemplo:

```
await conexao.beginTransaction();

try {
```

```
    await conexao.query("INSERT INTO usuarios(nome) VALUES(?)", ["Gabriel"]);

    await conexao.commit();

} catch (erro) {

    await conexao.rollback();

}
```

Isso garante a integridade dos dados.

9. Quais erros podem ocorrer ao conectar ao MySQL e como tratá-los no Node.js?

Alguns erros comuns são:

- usuário ou senha incorretos;
- banco inexistente;
- servidor MySQL desligado;
- timeout de conexão;
- perda de conexão.

O tratamento pode ser feito utilizando **try/catch**:

```
try {

    const conexao = await mysql.createConnection(config);

} catch (erro) {

    console.error("Erro ao conectar:", erro.message);

}
```

}

10. Por que é importante organizar o código em módulos ao trabalhar com bancos de dados?

A modularização torna o projeto mais organizado e facilita:

- reutilização de código;
- manutenção;
- testes;
- escalabilidade;
- separação de responsabilidades.

Um exemplo é separar:

projeto/

|

|— db.js

|— usuarioModel.js

|— usuarioController.js

|— routes.js

└— app.js

Cada arquivo possui uma responsabilidade específica, tornando o projeto mais limpo e profissional.